Distributed LLM Fine-Tuning & Inference on HPC Systems

NRIS

Norwegian research infrastructure services

Hicham Agueny, PhD
Scientific Computing Group
Info. Tech. Department, UiB/NRIS

Day 2 —

Distributed Training & Optimized Inference

09:30 – 10:15 | Distributed Training Concepts

- DDP vs FSDP
- Communication overhead and scaling efficiency

Hands-On Session – Practical Multi-GPU Fine-Tuning & Inference

Multi-GPU Fine-Tuning (Alpaca dataset)

10:30 – 11:15 | Multi-GPU LoRA fine-tuning
11:15 – 12:00 | Multi-GPU QLoRA fine-tuning

Inference: Question-Answering & Summarization

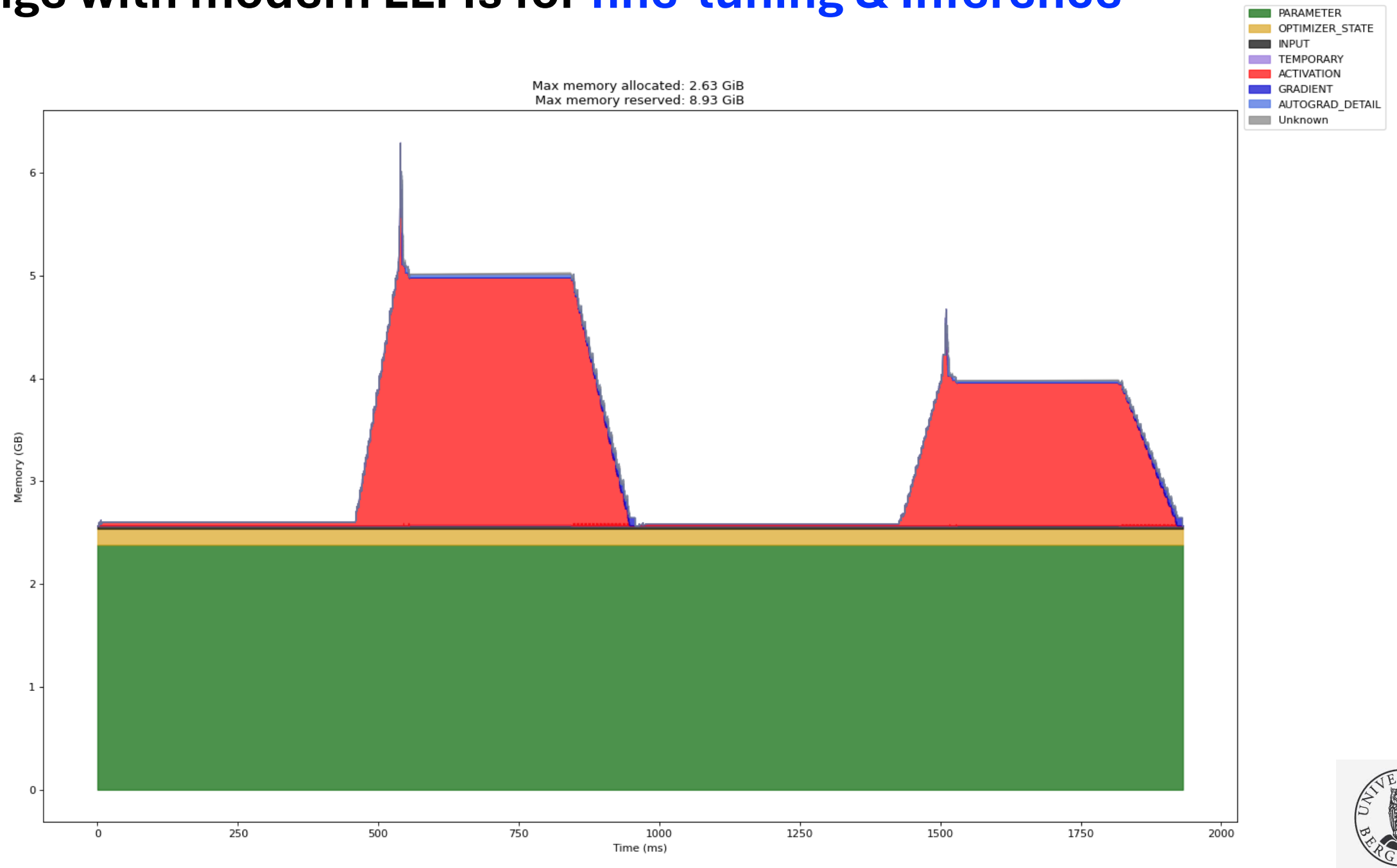
13:00 – 13:30 | Introduction to the vLLM inference engine
13:30 – 14:15 | Single-GPU inference benchmarking (LoRA & QLoRA)
14:30 – 15:30 | Multi-GPU inference (LoRA & QLoRA)

15:30 – 16:00 | Wrap-up & Discussion

Learning Outcomes

- Scale up fine-tuning to multiple GPUs with LoRA, QLoRA & QAT-LoRA.
- Scale up fine-tuning to multiple nodes with LoRA, QLoRA & QAT-LoRA
- Perform distributed inference & serving with vLLM.
- Monitor GPU usage and memory utilization & Profiling.

Challenge with modern LLMs for fine-tuning & Inference



Challenge with modern LLMs for fine-tuning & Inference

During fine-tuning, you need to store:

1. **Weights**: The model parameters themselves

2. **Gradients**: Computed during backpropagation for weight updates

3. **Optimizer States**: Adam maintains two estimates per parameter (m, v)

4. **Activations**: Intermediate outputs for backpropagation

Ex. Kimi 2.5 1 trillion parameters

FP32 (4 bytes): at least 1 TB x 4 x 4 = **16 TB**
FP16 (2 bytes): at least 1 TB x 2 x 4 = **8 TB**
FP4 (1/2 bytes): at least 1 TB x 0.5 x 4 = **2 TB**

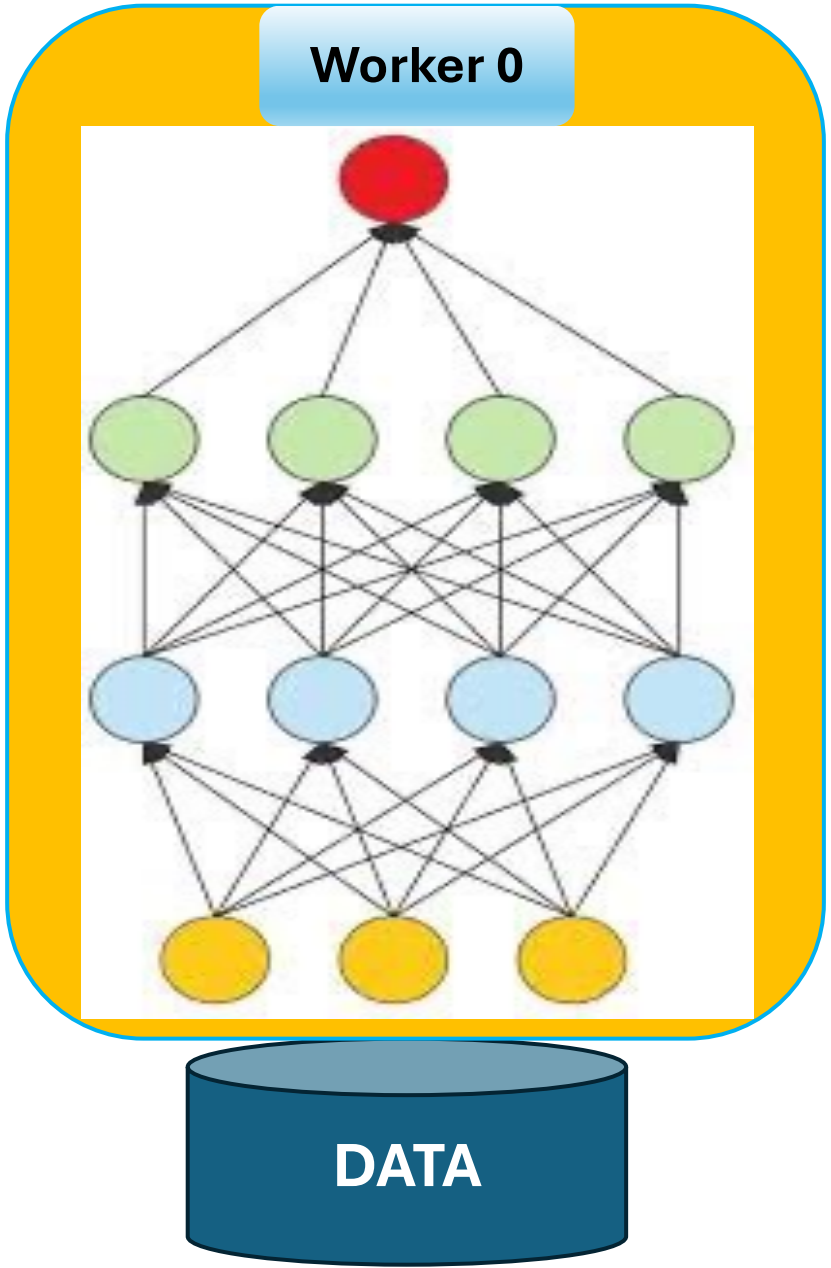
Model is too large to fit in a single GPU-memory

There is a need of a concept of parallelism to address the memory issue of large model

Concept of Distributed DNN Training

- **Model Parallelism**
- **Data Parallelism**
- **Concept FSDP – Fully Sharded Data Parallel**
- **Scalability**

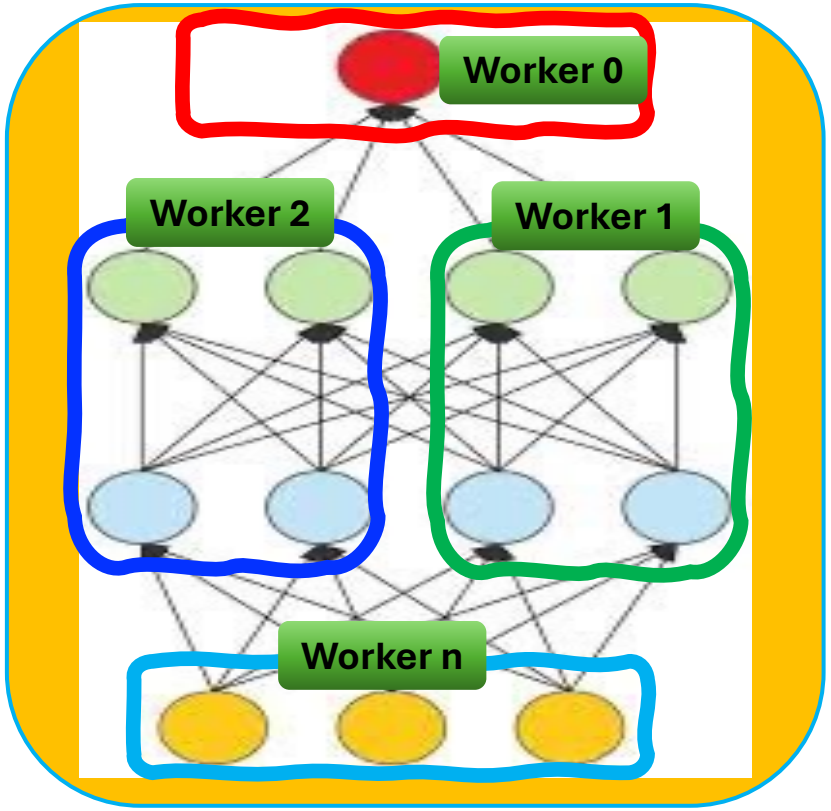
DNN Training on a single worker



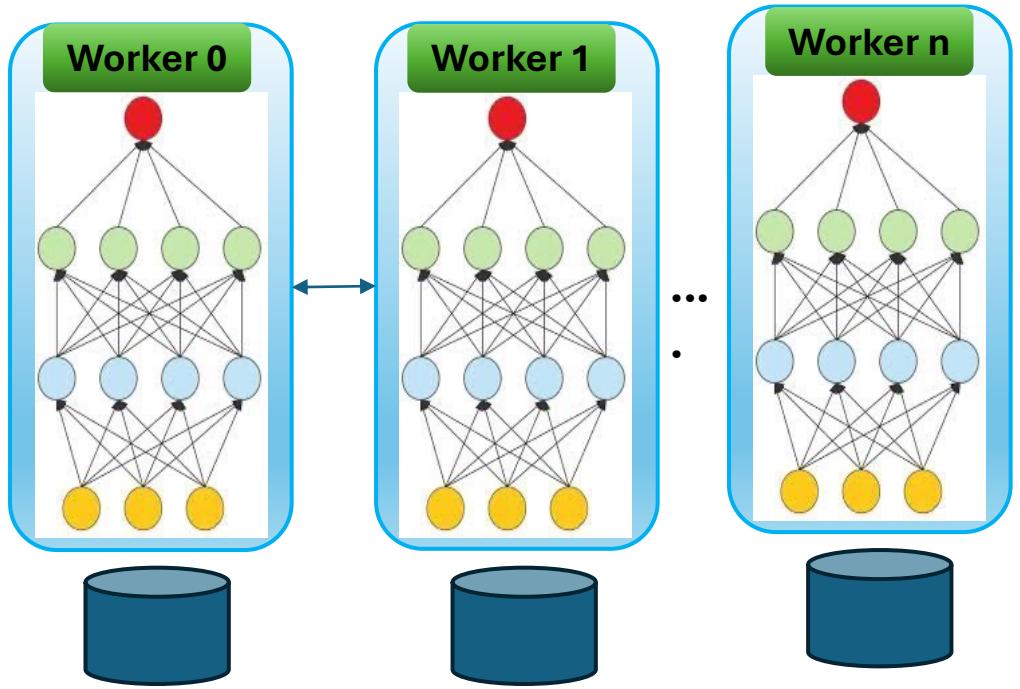
Distributed DNN Training: Parallelism Schemes

Parallelism in DNN: Training **large DNN models** or **large dataset** on **multiple Workers** in a shared or distributed environment.

Model Parallelism



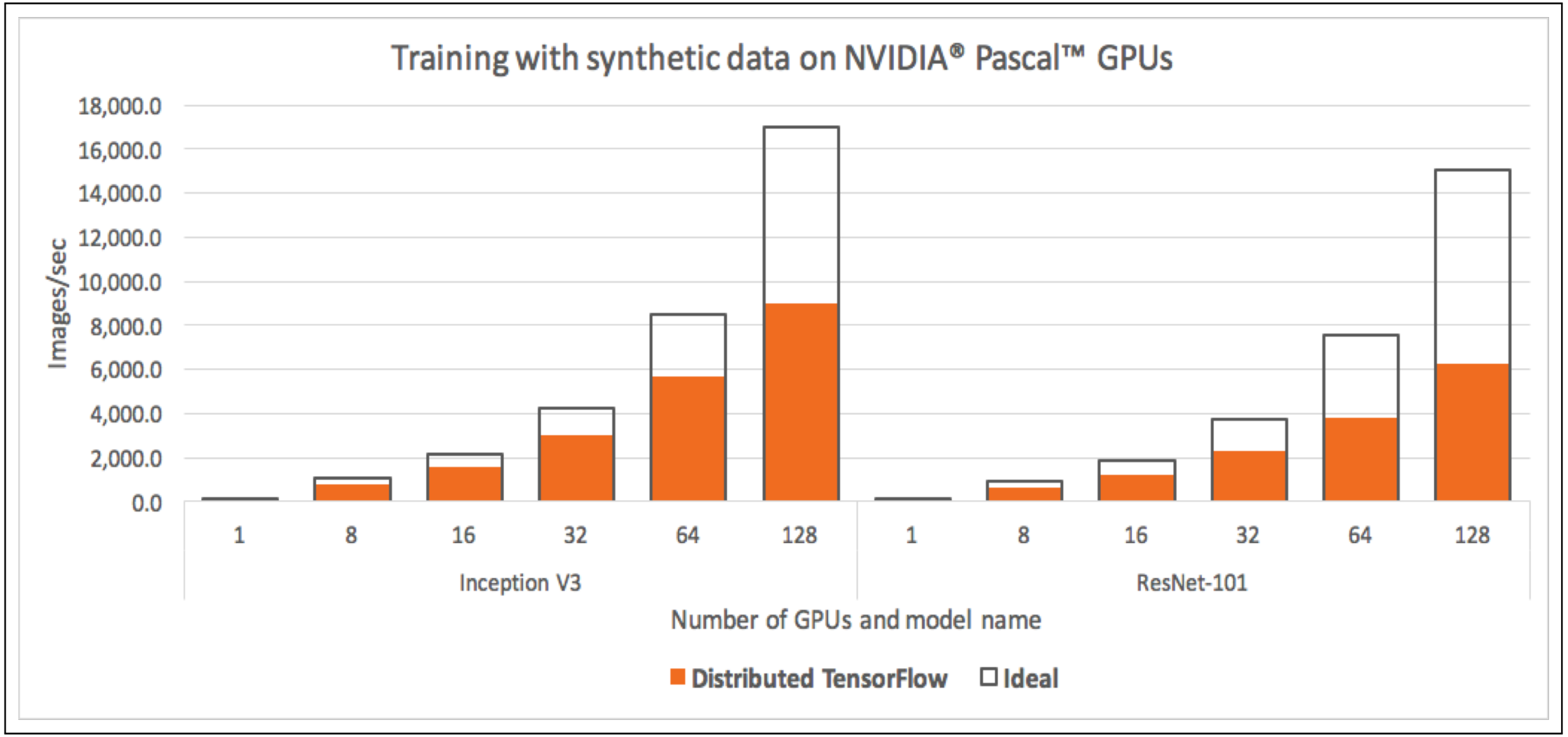
Data Parallelism



Simplicity
&
Scalability
&
Runtime Performance

Distributed TensorFlow – Model parallelism

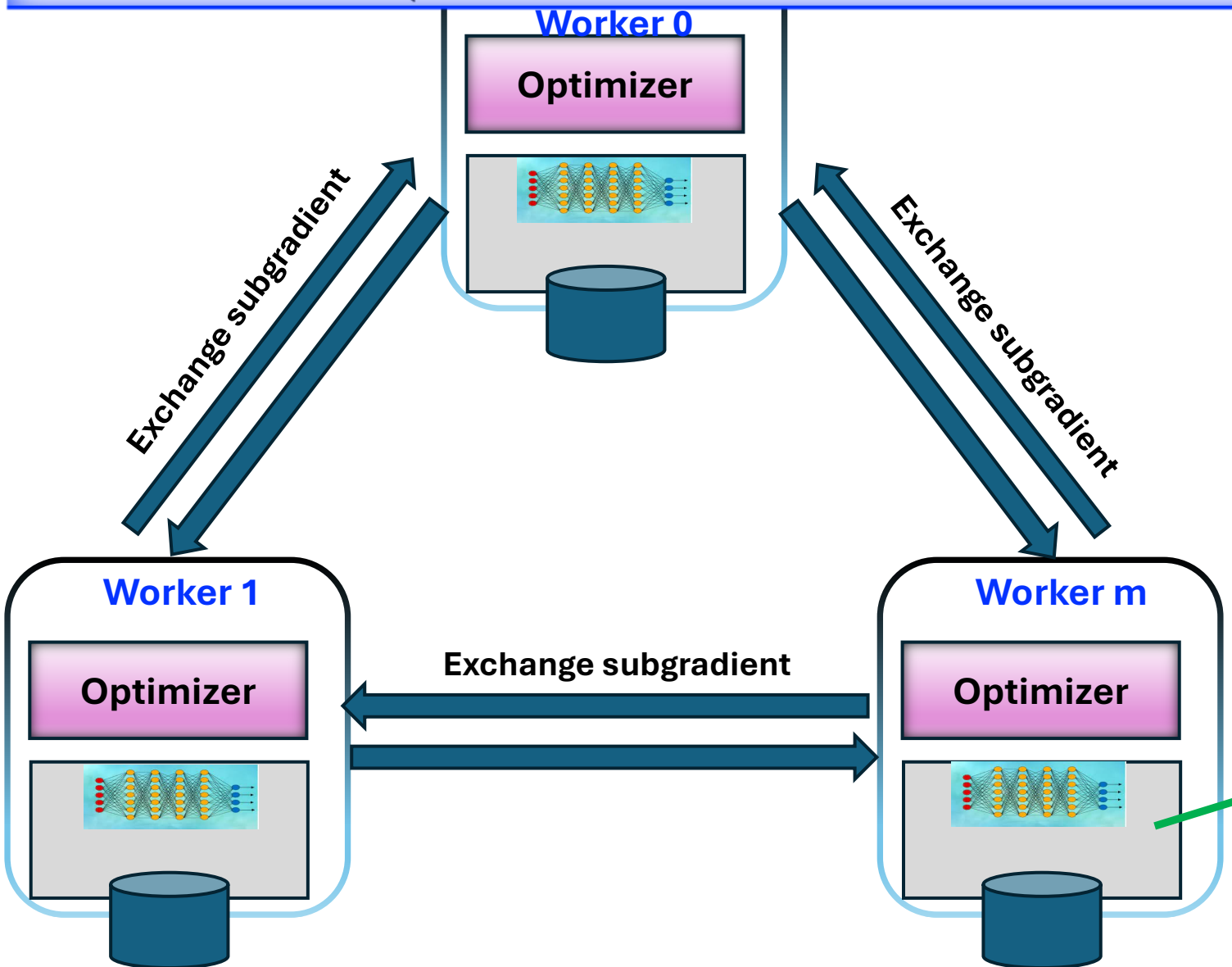
- **Poor Scaling (communication overhead)**



Alexander Sergeev, Mike Del Balso https://arxiv.org/abs/1802.05799

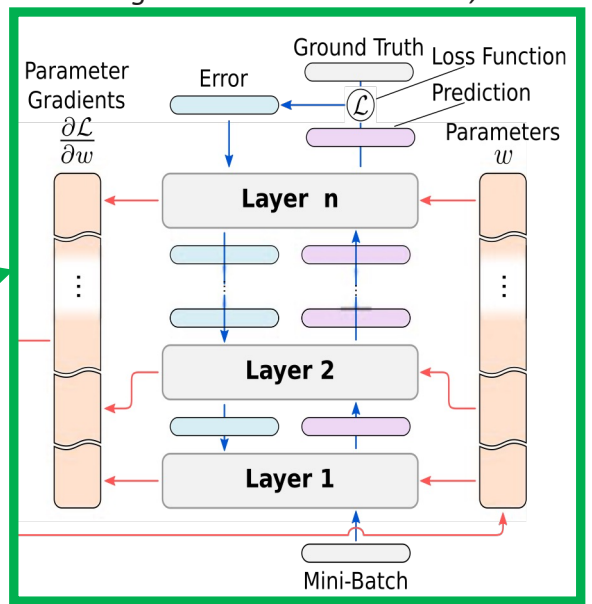
Decentralized Distributed DNN Training:

$$\nabla_{\mathbf{w}} f(\mathbf{w}; X) = \frac{1}{N}\left(\frac{1}{B}\sum_{i=1}^{B} \nabla_{\mathbf{w}} \ell(\mathbf{w}, \mathbf{x}_i) + \frac{1}{B}\sum_{i=B+1}^{B\times 2} \nabla_{\mathbf{w}} \ell(\mathbf{w}, \mathbf{x}_i) + \ldots + \frac{1}{B}\sum_{i=B\times(N-1)+1}^{B\times N} \nabla_{\mathbf{w}} \ell(\mathbf{w}, \mathbf{x}_i) \right)$$



- Each worker computes (sub)gradient and exchange its value with the neighbooring workers (forming a Ring)
- Each worker computes their own weights
- Each worker don't exchange weights with other workers

M. Langer et al. IEEE TPDS 31:12, 2020



Limitation of DDP – Distributed Data Parallel

- Each device holds a complete replica of parameters
- Model must be fully materialized on each GPU
- Initialization requires real tensor allocation in GPU memory
- Fails for very large models (cannot fit on a single GPU)

What's needed: a solution that achieves the performance of DDP and addresses the issue of scalability of large models

One of the solution is FSDP – Fully Sharded Data Parallel

Concept of PyTorch FSDP - Fully Sharded Data Parallel

FSDP is a distributed training approach designed to train Large deep learning models that exceed the memory capacity of a single GPU.

Core idea: Sharding (split & distribute) model parameters, gradients, and optimizer states across available GPUs.

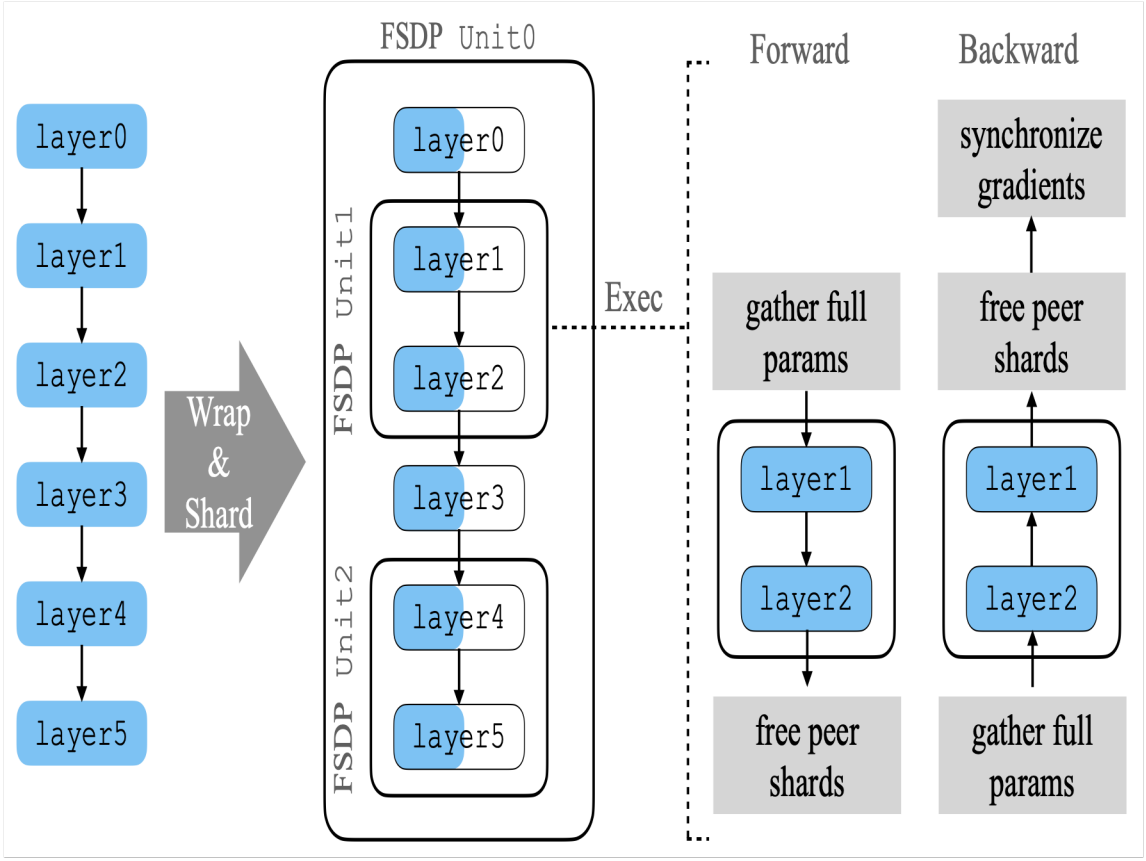
"Sharding" here means: **Splitting tensors into pieces and distribute those pieces across GPUs instead of replicating the full tensor on every GPU.**

The Sharding Mechanism

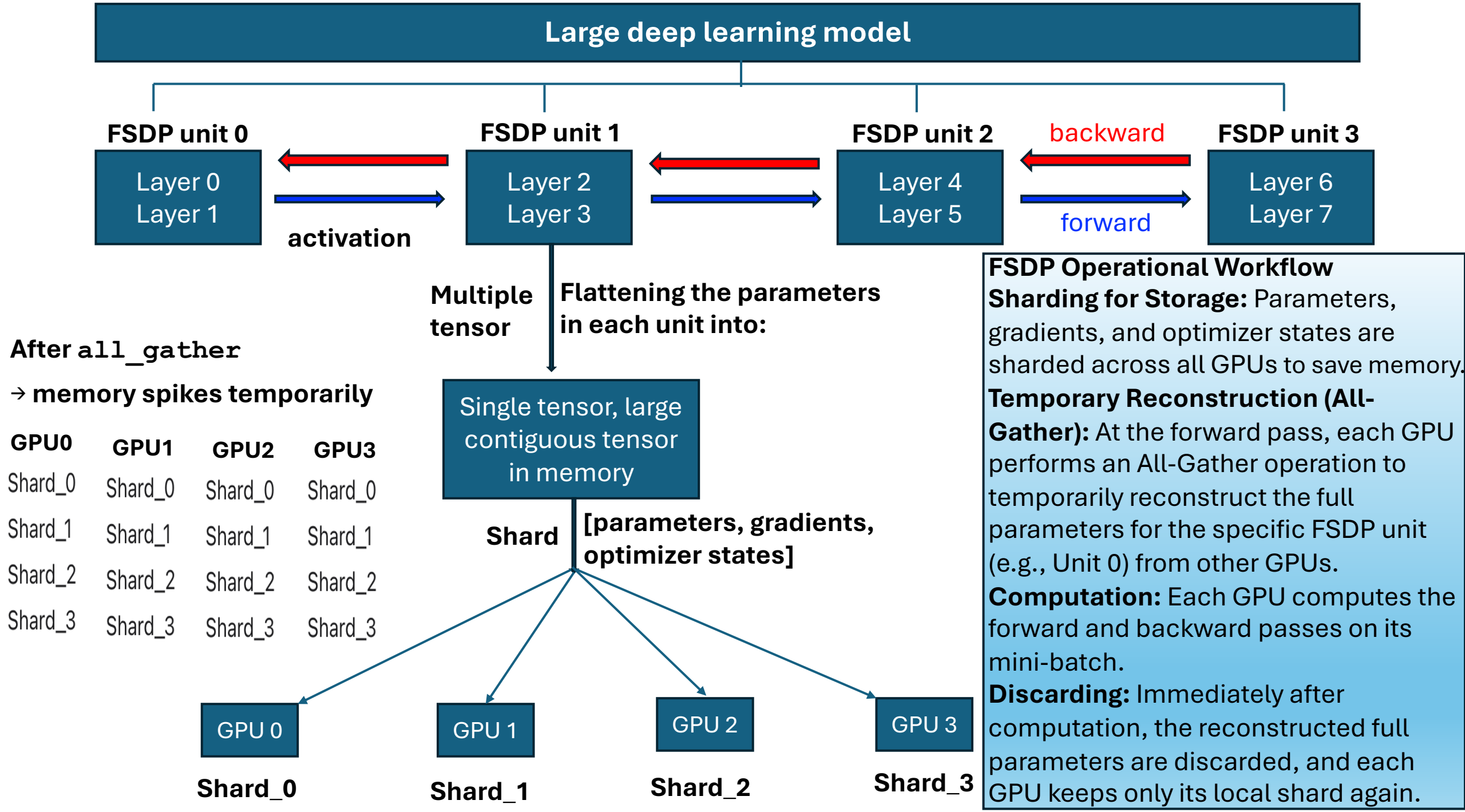
Decomposition: The model is broken down into smaller components called **FSDP units**.

Flattening & Sharding: Parameters within each unit are flattened and distributed across all available devices.

On-Demand Recovery: Parameters are temporarily "unsharded" (gathered) only when needed for computation. Peer shards are **immediately discarded** after use to minimize peak memory consumption.

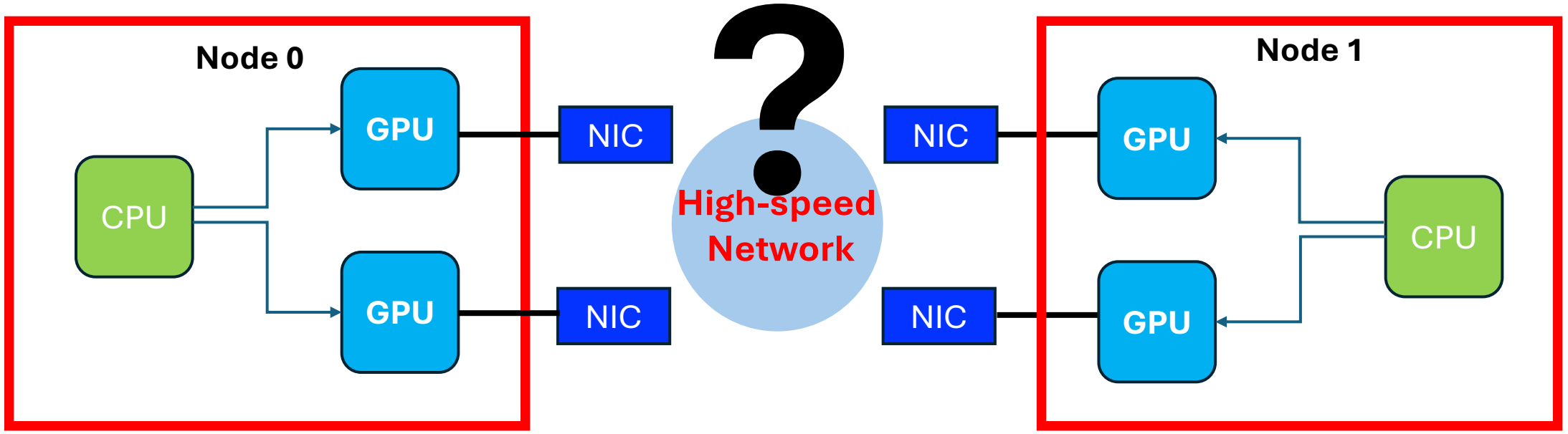


https://arxiv.org/abs/2304.11277


Large deep learning model
FSDP unit 0
Layer 0
Layer 1
activation
FSDP unit 1
Layer 2
Layer 3
FSDP unit 2
Layer 4
Layer 5
backward
forward
FSDP unit 3
Layer 6
Layer 7
Multiple tensor
Flattening the parameters in each unit into:
Single tensor, large contiguous tensor in memory
Shard
[parameters, gradients, optimizer states]
After all_gather
→ memory spikes temporarily
GPU0 GPU1 GPU2 GPU3
Shard_0 Shard_0 Shard_0 Shard_0
Shard_1 Shard_1 Shard_1 Shard_1
Shard_2 Shard_2 Shard_2 Shard_2
Shard_3 Shard_3 Shard_3 Shard_3
GPU 0
GPU 1
GPU 2
GPU 3
Shard_0
Shard_1
Shard_2
Shard_3
FSDP Operational Workflow
Sharding for Storage: Parameters, gradients, and optimizer states are sharded across all GPUs to save memory.
Temporary Reconstruction (All-Gather): At the forward pass, each GPU performs an All-Gather operation to temporarily reconstruct the full parameters for the specific FSDP unit (e.g., Unit 0) from other GPUs.
Computation: Each GPU computes the forward and backward passes on its mini-batch.
Discarding: Immediately after computation, the reconstructed full parameters are discarded, and each GPU keeps only its local shard again.

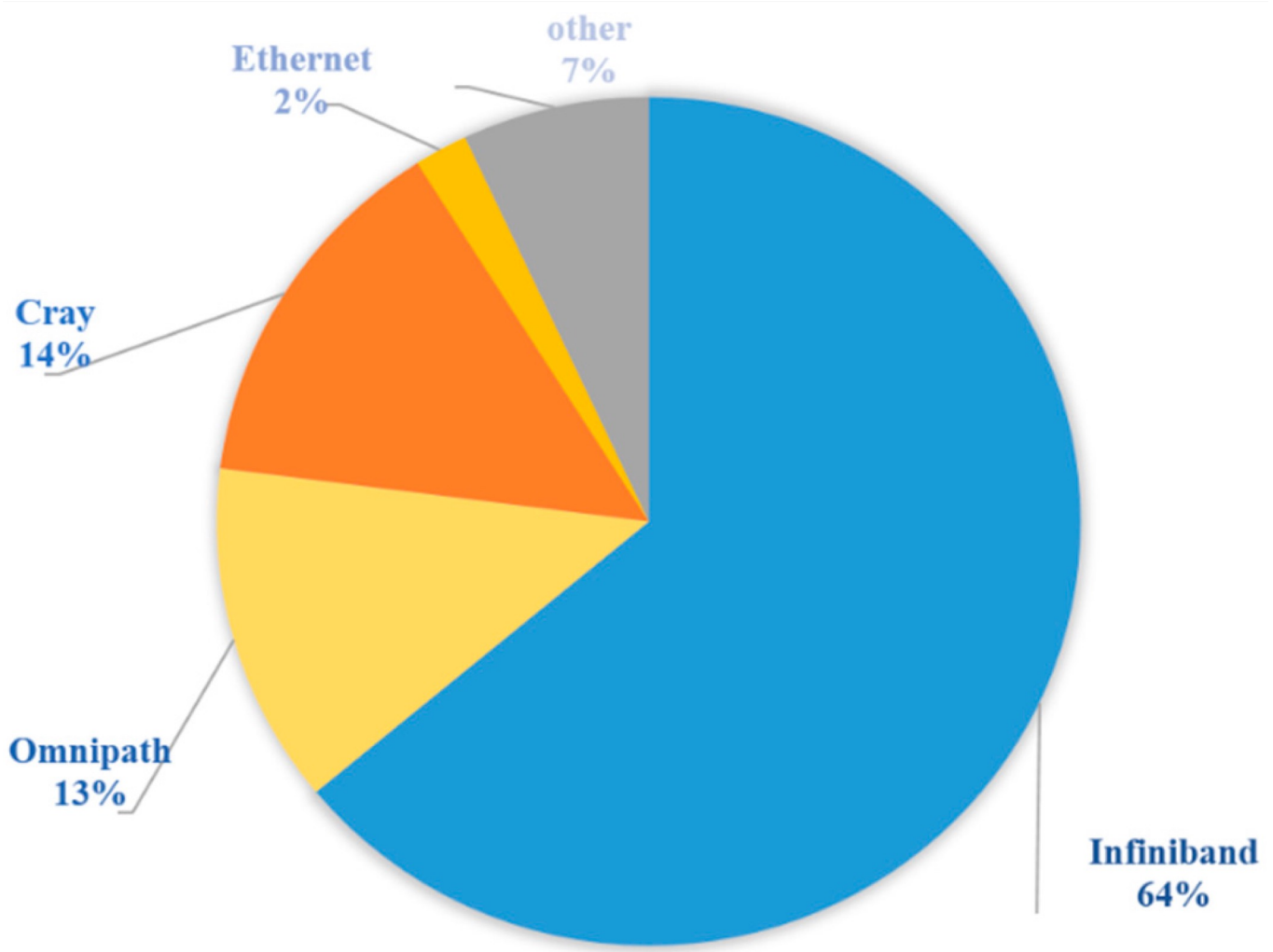
Communication

Network Interface Card - NIC



How does a compute node connect to thousands of other compute nodes in an HPC system ??

high-performance interconnects - HPIs

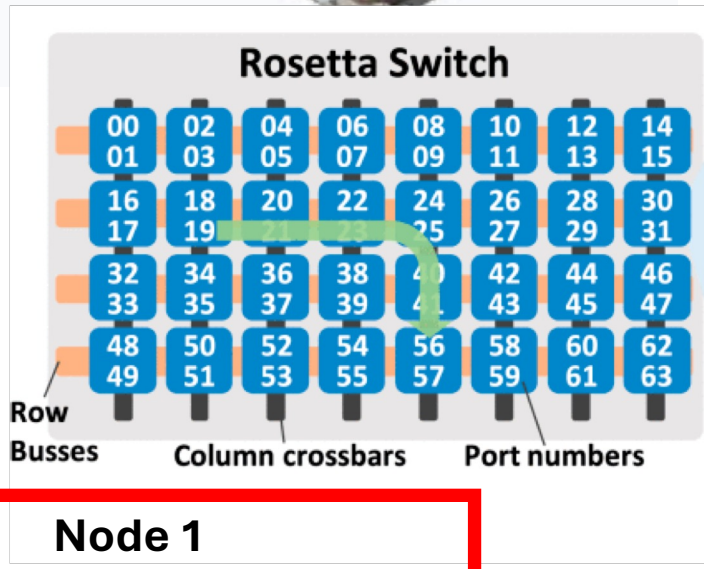


Distribution map of HPIs in the Top100 as of Nov 2021

Electronics 2022, **11**, 1369 10.3390/electronics11091369

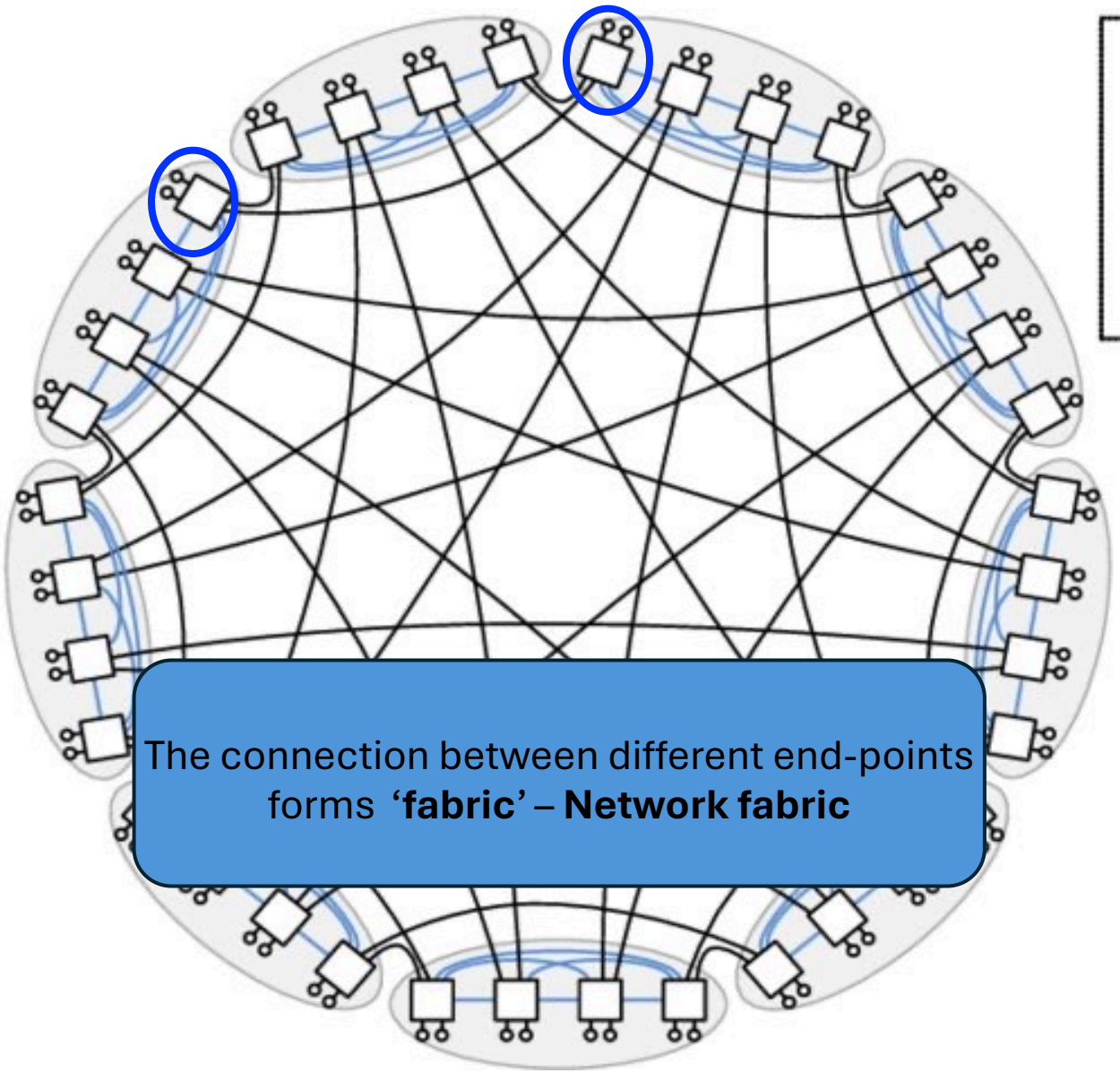
HPE Slingshot interconnects

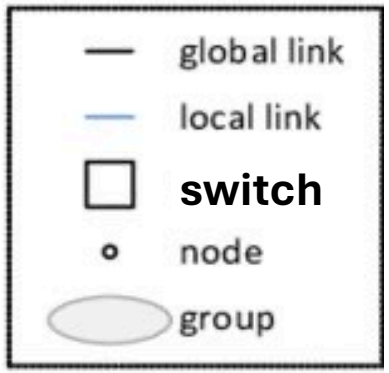
- **HPE Slingshot networks** are constructed from two components:
  - **Cassini – a** PCIe Gen4 Network Interface cards (NIC)
  - **Rosetta – a** 64-port **200 Gbps Ethernet switch** (Slingshot Switch)



- HPE Slingshot uses **Dragonfly topology to ensure**
  - High-bandwidth
  - Low latency
  - Scalability
  - Cost effectiveness

Dragonfly topology: Example of 72 end-points (compute nodes)





Dragonflies are organized as groups of switches.

Switches form a network that allows any node to reach any other node.

Ex. 9 groups
Each has 4 switches
Each switch connects 2 compute nodes
Total of 9x4x2 = **72 end-points (compute nodes)**

A compute node can communicate with any other node in at most 3 hops (steps)

1. **Node A** (Source) → **Switch 1** (1st hop)
2. **Switch 1** → **Switch 2** (2nd hop)
3. **Switch 2** → **Node B** (Destination)

Figure adapted from: https://ieeexplore.ieee.org/document/6337589

Scalability

Scalability

Np: number of GPUs

T_single: execution time on a single GPU

Speedup: $$S = \frac{T_{single}}{T_{multi}}$$

T_multi: execution time on multiple GPUs

Parallel efficiency $$P_e = \frac{1}{N_p}\frac{T_{single}}{T_{multi}}$$

Only LoRA

Setup	Time	Steps	Time/step	Efficiency
1 GPU	41m32s	26001	96ms	100%
4 GPU	18m44s	6500	173ms	55%
8 GPU	10m08s	3250	187ms	51%

1B model, doesn't benefit from multi-GPU training

Setup	Tokens/sec/GPU	Total Tokens/sec	Speedup vs 1 GPU	Per-GPU Efficiency
1 GPU	4447	4447	1.0×	100%
4 GPU	1316	5260	1.18×	29%
8 GPU	1144	9152	2.06×	26%

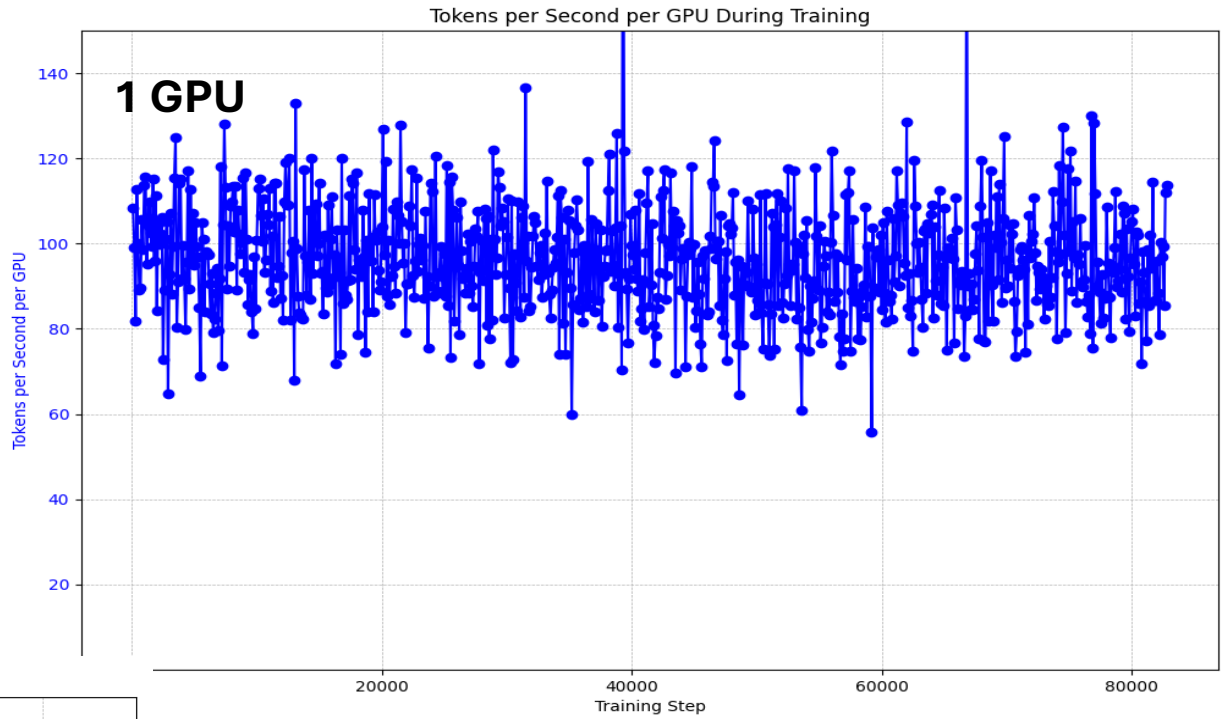
Throughput per-GPU:

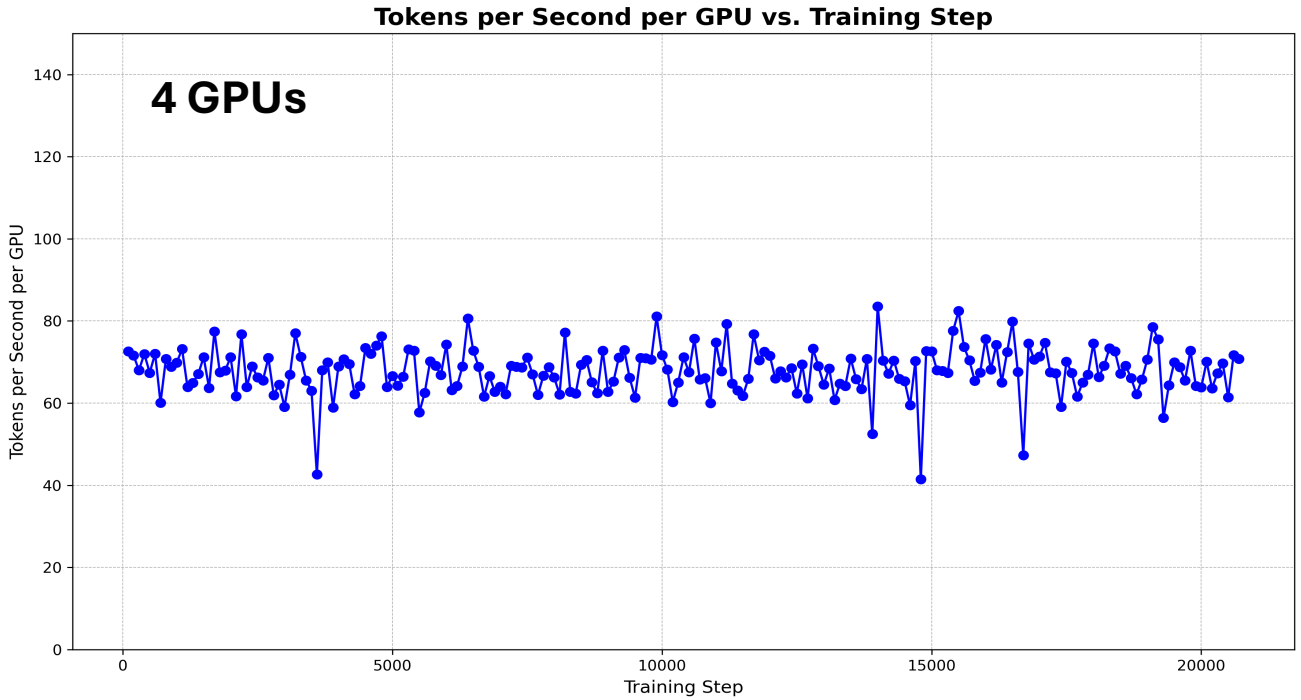
Model: **Llama 3.2-11B**

1GPU: up to **110** Tokens per sec per GPU
4 GPUs: up to **70** Token per sec per GPU

Total system throughput : 4x70=280

Runtime (1GPU): 1.1 second/training step (25.20 h)
Runtime (4 GPUs): 0.4 s/training step (9.04 h)





Scaling efficiency:
Parallel efficiency of ≈ 70% on 4 GPUs

Using **4 GPUs** reduced the training time
by a **factor of 2.8**

For much larger models, where the model itself cannot fit on a single GPU.....

Scalability - benchmarking

- Include warm-up steps
  - Run 2 initial steps as warm-up
  - The first step involves:
    - Memory allocation, Kernel initialization, Compilation (slow down the timing)

- Standardize scalability tests
  - Measure performance over a fixed window of 5–10 training steps (post warm-up)
  - Compute the average execution time per step for consistency.

- Monitor throughput
  - Track tokens per second per GPU
  - Use as a metric to assess scaling efficiency

Focus: Question Answering task

Task1: Multi-GPU, single node Scaling Benchmark (fine-tuning 45 min)

Objective: Evaluate the performance scaling of the fine-tuning for LoRA, QAT-LoRA and QLoRA

Instructions:

1- Run the training job using **4 GPUs**.

2- For each setup, record the **Total Time**, **Steps** completed, and **Time/step,** and calculate **Efficiency**

3- Calculate the **Tokens/sec** (both per GPU and Total), **Speedup** (relative to the 1 GPU run), and **Per-GPU Efficiency**.

4- Present your findings in two tables (one for task 2, and the other for task 3): LoRA vs Q-LoRA vs QAT-LoRA.

Slurm jobs:
$cd llm-hpc-course/day2_multi_gpu/fine-tuning/**ft_single_node**
Ex. For **LoRA**: $**cd lora_slurm**
$**sbatch job_multiGPU_QA_LoRA.sh**

Checkpoints:
llm-hpc-course/results/checkpoints_out/**llama3_2_1B_lora_multi_device**
Logs:
llm-hpc-course/results/logs/**lora_finetune_1B_output_multi_device**

Focus: Question Answering task

Task2: Multi-GPU, multi nodes Scaling Benchmark (fine-tuning 45 min)

Objective: Evaluate the performance scaling of the fine-tuning for LoRA, QAT-LoRA and QLoRA

Instructions:

1- Run the training job using **8 GPUs**.

2- For each setup, record the **Total Time**, **Steps** completed, and **Time/step, and calculate Efficiency**

3- Calculate the **Tokens/sec** (both per GPU and Total), **Speedup** (relative to the 1 GPU run), and **Per-GPU Efficiency**.

4- Present your findings in two tables (one for task 2, and the other for task 3): LoRA vs QLoRA vs QAT-LoRA

Slurm jobs:
$cd llm-hpc-course/day2_multi_gpu/fine-tuning/**ft_multi_node**
Ex. For **LoRA**: $**cd lora_slurm**
$**sbatch** job_multiNode_QA_LoRA.sh

Checkpoints:
llm-hpc-course/results/checkpoints_out/**llama3_2_1B_lora_multi_node**
Logs:
llm-hpc-course/results/logs/**lora_finetune_1B_output_multi_node**

Task3: Scaling Benchmark (15 min)

Present your findings from the previous tasks in two tables as shown below. Focus only on LoRA or QAT-LoRA

Setup	Time	Steps	Time/step	Efficiency
1 GPU	41m32s	26001	96ms	100%
4 GPU	18m44s	6500	173ms	55%
8 GPU	10m08s	3250	187ms	51%

Setup	Tokens/sec/GPU	Total Tokens/sec	Speedup vs 1 GPU	Per-GPU Efficiency
1 GPU	4447	4447	1.0×	100%
4 GPU	1316	5260	1.18×	29%
8 GPU	1144	9152	2.06×	26%

Overview of vLLM inference engine

Overview of vLLM inference engine

Efficient Memory Management for Large Language Model Serving with *PagedAttention*

Woosuk Kwon[1,*] Zhuohan Li[1,*] Siyuan Zhuang[1] Ying Sheng[1,2] Lianmin Zheng[1] Cody Hao Yu[3]
Joseph E. Gonzalez[1] Hao Zhang[4] Ion Stoica[1]
[1]UC Berkeley [2]Stanford University [3]Independent Researcher [4]UC San Diego

What is it? vLLM is an Open-source library for Large Language Model (LLM) **inference** and **serving**.

Origin: Developed initially by researchers at UC Berkeley.

The Problem: LLM inference is traditionally **memory-bound**, not compute-bound.
Standard serving frameworks struggle with memory fragmentation, leading to small batch sizes and wasted GPU resources (static memory allocation).

The Solution: vLLM improves throughput by managing memory in non-contiguous blocks, eliminating fragmentation.

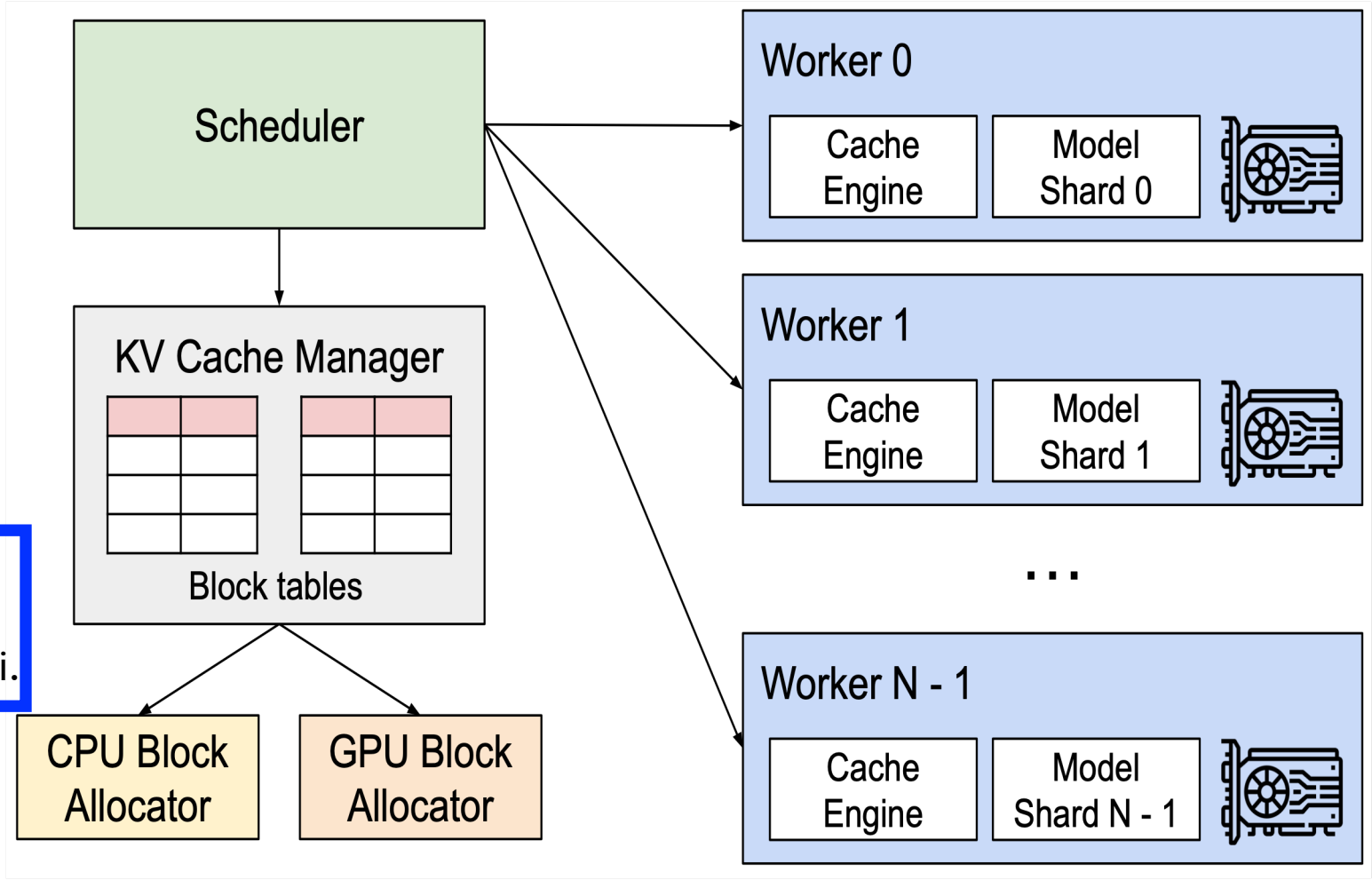
vLLM architecture

Scheduler:
To coordinate the execution of distributed GPU workers.

KV cache manager:
Manages the physical KV cache memory on the GPU workers through the instructions sent by the centralized scheduler.

KV cache dynamically grows and shrinks over time as the model generates new tokens, and its lifetime and length are not known a priori.

PagedAttention algorithm
Divides the request's KV cache into blocks, each of which can contain the attention keys and values of a fixed number of tokens.



https://arxiv.org/pdf/2309.06180

Key feature: "Continuous batching" - grouping a bunch of different user requests together to process them simultaneously and save compute time.

Quantization: supports heavily compressed models to further reduce the memory footprint.

Distributed Inference Strategy

1. Single GPU (No Distribution)

- Use when the model fits in one GPU
- Simplest setup, lowest overhead

2. Single Node, Multi-GPU (Tensor Parallelism)

- Use when the model doesn't fit in one GPU but fits in one machine
- Split model across GPUs within the same node
- *Tensor parallel size = number of GPUs (e.g., 4 GPUs → TP = 4)*

3. Multi-Node, Multi-GPU (Tensor + Pipeline Parallelism)

- Use when the model exceeds a single node's capacity
- Combine:
  - Tensor parallelism (within each node)
  - Pipeline parallelism (across nodes)
- *Example: 16 GPUs across 2 nodes → TP = 8, PP = 2*

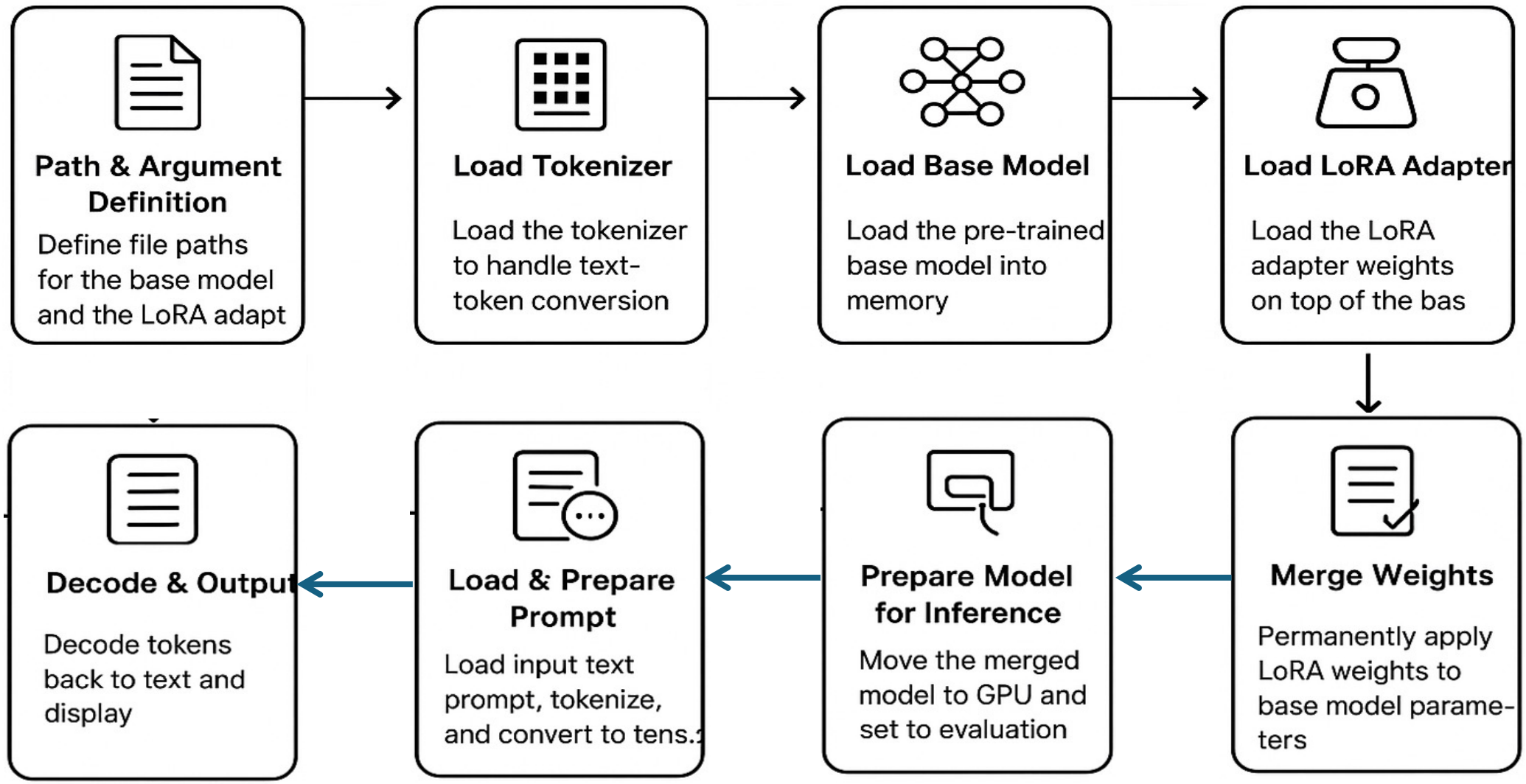
Scale from **single GPU → multi-GPU (single node) → multi-node** as model size exceeds hardware limits.

https://docs.vllm.ai/en/v0.8.1/serving/distributed_serving.html

Hands-on:

vLLM inference/serving: single GPU and multi GPUs

Structure of the Inference process



Task 1: Serving with vLLM – Quantified base model (no fine-tuning)

Step 1: Quantization

$cd llm-hpc-course/day2_multi_gpu/quantization

$sbatch job_singleGPU_QAT.sh

$tail –f out/qat-llama3-1B-1gpu-

Output of quantized mode:

$cd llm-hpc-course/shared/models/Llama-3.2-1B-Instruct-torchao_1GPU

Step 2: run serving on a single GPU

$cd llm-hpc-course/day2_multi_gpu/serving/single_device

Launch an interactive session: $**bash srun_sinteractive**

Launch a server: $**bash launch_server_qat-No-LoRA_nim.sh**

For GPU monitoring: Open a new terminal and ssh to the gpu-node: run the command **nvitop**

To provide a prompt: Open a new terminal and navigate to "**llm-hpc-course/day2_multi_gpu/serving/single_device**"

$**bash chat_no_lora.sh**

Prompt ex: Explain what is HPC ?

Task 2: Serving with vLLM – Fine-tuned model with LoRA

$cd llm-hpc-course/day2_multi_gpu/serving/single_device

Launch an interactive session: $**bash srun_sinteractive**

Launch a server: $**bash launch_server_LoRA_nim.sh**

For GPU monitoring: Open a new terminal and ssh to the gpu-node: run the command **nvitop**

To provide a prompt: Open a new terminal and navigate to "**llm-hpc-course/day2_multi_gpu/serving/single_device**"

$**bash chat_no_lora.sh**

Prompt ex: Explain what is HPC ?

Task 3: Serving with vLLM – Fine-tuned model with QAT-LoRA

$cd llm-hpc-course/day2_multi_gpu/serving/single_device

Launch an interactive session: $**bash srun_sinteractive**

Launch a server: $**bash launch_server_qat-LoRA_nim.sh**

For GPU monitoring: Open a new terminal and ssh to the gpu-node: run the command **nvitop**

To provide a prompt: Open a new terminal and navigate to "**llm-hpc-course/day2_multi_gpu/serving/single_device**"

$**bash chat_no_lora.sh**

Prompt ex: Explain what is HPC ?

Task 4: Serving with vLLM on multiple GPUs: LoRA

Step 1: run serving on 4 GPUs

$cd llm-hpc-course/day2_multi_gpu/serving/single_device

$ update to 4 gpus: --gpus=4 --ntasks-per-node=4

$**bash** launch_server_LoRA_nim.sh

For GPU monitoring: Open a new terminal and ssh to the gpu-node: run the command **nvitop**

To provide a prompt: Open a new terminal and navigate to "**llm-hpc-course/day2_multi_gpu/serving/single_device**"

$**bash chat.sh**

Prompt ex: Explain what is HPC ?

Output from vLLM

No Quantification: Only LoRA

```
(EngineCore_DP0 pid=1693230) INFO 06-09 22:06:44 [default_loader.py:308] Loading weights took 2.25 seconds
(EngineCore_DP0 pid=1693230) INFO 06-09 22:06:44 [punica_selector.py:20] Using PunicaWrapperGPU.
(EngineCore_DP0 pid=1693230) INFO 06-09 22:06:44 [gpu_model_runner.py:3549] Model loading took 2.4340 GiB memory and 3.776009 seconds
(EngineCore_DP0 pid=1693230) INFO 06-09 22:06:50 [backends.py:655] Using cache directory: /cluster/work/projects/nn9970k/hicham/llm-hpc-course/.cache/vllm/torch_com
pile_cache/e87956e9b4/rank_0_0/backbone for vLLM's torch.compile
```

Quantification of weights without LoRA

```
(EngineCore_DP0 pid=1656796) INFO 06-09 21:52:33 [default_loader.py:308] Loading weights took 1.26 seconds
(EngineCore_DP0 pid=1656796) INFO 06-09 21:52:33 [gpu_model_runner.py:3549] Model loading took 1.4130 GiB memory and 2.976190 seconds
(EngineCore_DP0 pid=1656796) INFO 06-09 21:52:38 [backends.py:655] Using cache directory: /cluster/work/projects/nn9970k/hicham/llm-hpc-course/.cache/vllm/torch_com
pile_cache/9a099cdca8/rank_0_0/backbone for vLLM's torch.compile
```

Quantization with LoRA

```
(EngineCore_DP0 pid=1766106) INFO 06-09 22:36:39 [gpu_model_runner.py:3549] Model loading took 1.5292 GiB memory and 3.172193 seconds
```

References

- **LoRA:** https://arxiv.org/pdf/2106.09685
- **LoRA Dropout:** https://arxiv.org/pdf/2404.09610
- **QLoRA:** https://arxiv.org/pdf/2305.14314
- **PEFT:** https://arxiv.org/abs/2312.12148
- **PyTorch FSDP:** https://arxiv.org/pdf/2304.11277
- **ZeRO:** https://arxiv.org/abs/1910.02054
- **Ultimate Guide to Fine-Tuning LLMs**: https://arxiv.org/pdf/2408.13296
- **Merging model**: https://huggingface.co/docs/peft/en/developer_guides/lora#weight-decomposed-low-rank-adaptation-dora
- **Transformer:** https://arxiv.org/pdf/1706.03762
- **Fastformer:** https://arxiv.org/pdf/2108.09084
- **vLLM-PagedAttension:** https://arxiv.org/pdf/2309.06180
- **vLLM examples:** https://docs.vllm.ai/en/v0.11.0/examples/offline_inference/data_parallel.html
- **PyTorch container**: https://docs.nvidia.com/deeplearning/frameworks/pytorch-release-notes/rel-24-09.html
- **GitHub repo LLM-course:** https://github.com/HichamAgueny/llm-hpc-course